

Estrategias de Ramificación en Git

Guía Completa de Flujos de Trabajo (Branching Workflows) y Casos de Uso

En Git, la forma en la que un equipo gestiona sus ramas define su ritmo de desarrollo, la estabilidad de su código y la velocidad con la que lanzan cambios al mercado. No existe una estrategia única perfecta; la elección depende estrictamente de la naturaleza del software y la madurez del equipo.

1. GitHub Flow

PROYECTOS ÁGILES Y DESPLIEGUE CONTINUO

Ideal para: Startups, Aplicaciones Web, Entornos SaaS

Es un modelo ágil, sencillo y centrado en la entrega constante de valor. La premisa fundamental es que la rama principal siempre está limpia y lista para ser desplegada en producción.

Las Ramas:

- **main (o master):** Es la rama principal. Contiene código 100% estable, protegido y en producción. Nadie programa directamente aquí.
- **Ramas de características (feat/* , fix/*):** Ramas temporales de corta duración que se abren exclusivamente para desarrollar una tarea específica o resolver un bug.

Flujo de trabajo paso a paso:

1. Creas una rama local nueva desde **main** escribiendo

```
git checkout -b feat/login-google
```

2. Trabajas en tu código haciendo commits pequeños y frecuentes.
3. Subes la rama al servidor remoto y abres un **Pull Request (PR)** para que tus compañeros revisen, comenten y aprueben el código.
4. Una vez aprobado el PR y superadas las pruebas o tests automáticos de integración, la rama se fusiona (*merge*) de vuelta a **main**.
5. El código se despliega automáticamente a producción y la rama temporal se elimina por completo.

2. GitFlow

SOFTWARE TRADICIONAL Y VERSIONES PROGRAMADAS

Ideal para: Apps Móviles (iOS/Android), Software embebido o Corporativo con hitos fijos

Un modelo robusto, rígido y altamente estructurado diseñado para proyectos que publican lanzamientos de versiones periódicas bien definidos (ej. v1.0, v2.0) y necesitan pasar por rigurosos filtros de calidad.

Las Ramas Estructuradas:

- **main** : Solo contiene el historial de las versiones oficiales lanzadas al público final. Cada commit aquí es un hito de versión.
- **develop** : Es la rama central del día a día. Acumula las funciones terminadas que se integrarán en el siguiente lanzamiento oficial.
- **feature/*** : Ramas para programar nuevas funciones. Nacen siempre desde **develop** y mueren de vuelta en **develop**.
- **release/*** : Colchón de preparación de versiones. Cuando **develop** tiene lo necesario para una entrega, se abre una rama **release/v1.1** para testear a fondo y corregir pequeños detalles antes del lanzamiento final.
- **hotfix/*** : Ramas de emergencia. Si surge un fallo crítico en el código de producción (**main**), se crea un hotfix directo desde ahí para solucionarlo rápido. Al terminar, se fusiona tanto en **main** como en **develop**.

3. Trunk-Based Development

EQUIPOS DE ALTO RENDIMIENTO E INTEGRACIÓN CONTINUA AVANZADA

Ideal para: Equipos senior, Grandes Tecnológicas (Google, Meta, Netflix)

Es el modelo totalmente opuesto a GitFlow. Busca eliminar el "dolor de integración" (los temidos conflictos al fusionar ramas que llevan semanas abiertas) obligando a los desarrolladores a integrar su código muchas veces al día en un único tronco común.

El Funcionamiento Diario:

- Rama **trunk** (o **main**): Concentra todo el trabajo. Todos los programadores suben código directamente aquí o mediante Pull Requests ultra rápidos que se resuelven en cuestión de horas.

- **Mecanismo de seguridad (Feature Flags):** ¿Cómo se sube código a medias sin romper la aplicación activa? Los desarrolladores usan "Feature Flags" (interruptores lógicos). La nueva funcionalidad se sube oculta detrás de un condicional estricto en el código (ej. `if (flag_activo)`) y solo se enciende de forma externa en el panel de control una vez completada y testeada.

Matriz de Decisión Rápida

Estrategia	Complejidad	Ritmo de Despliegue	Foco de Control
GitHub Flow	Baja	Continuo (Múltiples veces al día)	Pull Requests sencillos
GitFlow	Alta	Ciclos / Versiones fijas	Estabilidad estricta
Trunk-Based	Media-Alta	Constante / Automatizado	Automatización y Tests

Conclusiones para el Día a Día

Para proyectos web estándar o si trabajas solo/en un equipo pequeño que necesita agilidad, adopta **GitHub Flow**. Si tu producto se distribuye por versiones inamovibles o pasa auditorías rigurosas, asume la complejidad de **GitFlow**. Si el equipo es maduro, tiene una cobertura de tests automáticos del 100% y busca la máxima velocidad de ingeniería, evoluciona hacia **Trunk-Based Development**.

Documento completo generado para Brais • Guía visual expandida de flujos de trabajo en Git.